

Algorithms Final

Q-1) (50 points) Given a rod of length n inches and an array $A[1...n]$ of prices that includes prices of all pieces of discrete sizes smaller than or equal to n . Determine the maximum value obtainable by cutting up the rod and selling the pieces. For example, if the length of rod is 8 and the values of different pieces are given as the following, then the maximum obtainable value is 22 (by cutting in two pieces of lengths 2 and 6)

length	1	2	3	4	5	6	7	8

price	1	5	8	9	10	17	17	20

And if the prices are as following, then the maximum obtainable value is 24 (by cutting in eight pieces of length 1)

length	1	2	3	4	5	6	7	8

price	3	5	8	9	10	17	17	20

a) (25 pts.) Give a mathematical recursive formulation for $R(n)$ which denotes the maximum possible price value you can get for a rod of length n .

b) (5 pts.) Show that this problem has the overlapping subproblems property.

c) (15 pts.) Write a dynamic programming algorithm (as a pseudocode) for finding the maximum price that can be obtained for a rod of length n .

d) (5 pts.) Provide the running time of your dynamic programming algorithm. Explain briefly.

Q-2) (20 pts.) You are given an array of jobs where every job takes single (i.e.1) unit of time. Also, every job has a deadline (i.e. latest time or date by which something should be completed) and an associated profit if the job is finished before the deadline. Thus, the minimum possible deadline for any job is 1. If only one job can be scheduled at a time, the job sequencing problem is to maximize total profit. Below is an example optimal solution for the given job list:

Input: Five jobs with the following deadlines and profits

Output: The maximum profit Job sequence of jobs is a, c, e.

JobID	Deadline	Profit
a	2	100
b	1	19
c	2	27
d	1	25
e	3	15

a) (10 pts.) Describe, in words, a greedy algorithm for the job sequencing problem.

b) (10 pts.) Prove that your greedy choice is optimal (e.g. proof by contradiction).

Q-3) (20 pts.) Assume you are creating an array data structure that has a fixed size of n . You want to backup this array after every n insertion operations. Unfortunately, the backup operation is quite expensive, it takes n time to do backup. Insertions without a backup just take 1 time unit. Show that you can do backups in $O(1)$ amortized time. Use potential method.

Q-4) (10 pts.) Given an unsorted array, the array has this property that every element in array is at most k distance from its position in sorted array where k is a variable smaller than size of array. For example, let us consider k is 2, an element at index 7 in the sorted array, can be at indexes 5, 6, 7, 8, 9 in the given array. We can sort such arrays more efficiently with the help of Heap data structure. Following is the sorting algorithm that uses a Min-Heap:

- Create a Min-Heap of size $k+1$ with first $k+1$ elements.
- One by one remove min elements from heap, put it in result array, and add a new element to heap from remaining elements.

Provide a tight asymptotic upper bound for this algorithm. Explain briefly.